PARTE 2

RottenPotatoes: Git e GitHub no RubyMine

Do projeto local sem repositório ao fluxo colaborativo com branches e pull requests

PONTO DE PARTIDA	Aplicação RottenPotatoes pronta localmente, ainda sem repositório Git.
FOCO	Versionamento, publicação no GitHub, colaboração e rastreabilidade.
ENTREGA	Repositório publicado, branch de atividade, issue, pull request revisado e mergeado.

Resultado esperado

Ao final, cada aluno compreenderá a diferença entre salvar localmente e publicar remotamente, saberá revisar mudanças antes do commit e usará um pull request para integrar trabalho à branch main.

ETAPA 0

Objetivos e fluxo de trabalho

Nesta parte, Git registra versões no computador e GitHub hospeda e organiza a colaboração. O RubyMine fornece a interface visual, mas os comandos equivalentes serão apresentados para tornar cada operação compreensível.

Objetivos de aprendizagem

- Inicializar um repositório a partir de um projeto Rails existente.
- Distinguir arquivo modificado, área de preparação, commit e repositório remoto.
- Publicar o projeto no GitHub sem expor arquivos sensíveis.
- Criar branches, commits pequenos e mensagens descritivas.
- Relacionar issue, branch e pull request.
- Revisar diferenças, integrar mudanças e sincronizar a branch main.

Fluxo que será praticado

Editar	Revisar diff	Commit	Push	Pull request	Merge	Pull

Regra da aula

Antes de cada commit ou push, o aluno deve abrir o diff, explicar o que mudou e conferir se nenhum segredo ou arquivo gerado foi incluído.

ETAPA 1

Entender Git, GitHub e os estados de um arquivo

Git e GitHub participam do mesmo fluxo, mas não são a mesma coisa.

No RubyMine / GitHub	Comando ou resultado
Git	Controla versões localmente: alterações, commits, branches e histórico.
GitHub	Hospeda o repositório remoto e oferece issues, pull requests e revisão.
RubyMine	Exibe mudanças, diferenças, histórico e executa operações Git/GitHub.

Estados principais

- Não rastreado: o arquivo existe, mas o Git ainda não o acompanha.
- Modificado: o arquivo mudou desde o último commit.
- Preparado (staged): a versão escolhida está pronta para o próximo commit.
- Commitado: a versão foi registrada no histórico local.
- Publicado: os commits locais foram enviados ao repositório remoto.

Modelo mental

```
arquivo de trabalho -> stage -> commit local -> push -> GitHub
```

Commit não é sinônimo de backup remoto. Um commit existe localmente até que seja enviado com push.

ETAPA 2

Confirmar e inicializar o repositório local

Abra o RottenPotatoes no RubyMine. Como o exercício supõe que não existe repositório, confirme primeiro no terminal integrado:

```
git status
```

Se aparecer a mensagem de que não é um repositório Git, inicialize pela interface:

No RubyMine / GitHub	Comando ou resultado
VCS > Enable Version Control Integration	Selecione Git e confirme.
Terminal integrado	Alternativa: executar git init na raiz rottenpotatoes.
Git tool window	Confirme que os arquivos aparecem como mudanças não rastreadas.

Comando equivalente

```
git init
git status
```

Caso git status já funcione

Algumas versões do comando rails new iniciam Git automaticamente. Nesse caso, não execute git init novamente: continue usando o repositório existente e inspecione seu histórico.

Verificação

[ ] A raiz Git corresponde à pasta rottenpotatoes.

[ ] O RubyMine passou a mostrar o widget da branch.

[ ] git status lista os arquivos do projeto.

ETAPA 3

Configurar identidade e proteger arquivos

O autor de cada commit é definido por nome e e-mail. Em laboratório compartilhado, prefira configuração local, válida somente neste repositório:

```
git config user.name "Nome do Aluno"
git config user.email "email-usado-no-github@example.com"

git config --get user.name
git config --get user.email
```

Revisar .gitignore

Abra o arquivo .gitignore na raiz do projeto. Ele deve excluir arquivos temporários, logs, bancos locais e segredos. A lista exata varia conforme a versão do Rails.

```
/log/*
/tmp/*
/storage/*
/config/master.key
.env*
/db/*.sqlite3*
```

Não copie cegamente esta lista sobre um .gitignore existente. Compare e acrescente somente o que estiver ausente e for compatível com o projeto.

Nunca publicar

- senhas e tokens
- config/master.key
- arquivos .env com credenciais
- chaves privadas
- bancos locais com dados reais

Antes do primeiro commit, use a janela Commit para verificar individualmente todos os arquivos selecionados.

ETAPA 4

Criar o primeiro commit

Abra a janela Commit do RubyMine. Selecione os arquivos adequados, abra o diff e confirme que o projeto inicia e que os testes passam.

Verificações antes do commit

[ ] Nenhuma credencial está selecionada.

[ ] Arquivos temporários e banco SQLite não estão selecionados.

[ ] As views Haml e o CSS preservam a indentação.

[ ] bin/rails test termina sem falhas.

Mensagem sugerida

```
Cria aplicação RottenPotatoes inicial
```

Pela interface

No RubyMine / GitHub	Comando ou resultado
Commit tool window	Marque os arquivos, revise o diff e escreva a mensagem.
Commit	Registre a versão no repositório local.
Git > Show History	Confirme que o commit aparece no histórico.

Comandos equivalentes

```
git add .
git commit -m "Cria aplicação RottenPotatoes inicial"
git log --oneline
```

git add . é apresentado como equivalente, mas na atividade o aluno deve selecionar e revisar os arquivos na interface para não adicionar conteúdo indevido.

ETAPA 5

Publicar um novo repositório no GitHub

Entre na conta do GitHub pelo RubyMine em Settings > Version Control > GitHub e use Log In via GitHub. Depois, com o projeto aberto:

No RubyMine / GitHub	Comando ou resultado
Git > GitHub > Share Project on GitHub	Abra o diálogo de publicação.
Repository name	Use rottenpotatoes ou o padrão definido pelo professor.
Remote	Mantenha origin.
Visibility	Prefira Private para atividades acadêmicas, salvo orientação contrária.
Share	Crie o remoto e envie o histórico local.

Caminho alternativo pelo site

Se o repositório for criado primeiro no GitHub, crie-o vazio: não adicione README, licença ou .gitignore, pois o projeto local já possui arquivos e histórico.

```
git branch -M main
git remote add origin URL_DO_REPOSITORIO
git push -u origin main
```

Não copie literalmente URL_DO_REPOSITORIO. Substitua pela URL HTTPS ou SSH exibida no repositório do aluno.

Verificação

[ ] O GitHub mostra os arquivos da aplicação.

[ ] O commit inicial aparece no histórico.

[ ] Nenhum segredo foi publicado.

ETAPA 6

Compreender origin, main, push e pull

Depois da publicação, o projeto possui uma cópia local e outra hospedada no GitHub.

No RubyMine / GitHub	Comando ou resultado
origin	Nome convencional do repositório remoto.
main	Branch principal usada como base de integração.
push	Envia commits locais ao remoto.
fetch	Busca informações remotas sem integrar o conteúdo à branch atual.
pull	Busca e integra mudanças remotas à branch local acompanhada.

Comandos de inspeção

```
git remote -v
git branch --show-current
git status
git log --oneline --decorate --graph --all
```

Experimento

Faça uma pequena alteração no README, crie um commit local e observe o indicador de commits de saída no widget Git. Só depois execute Git > Push e confira o novo commit no GitHub.

Antes do push, sincronize quando houver trabalho de outras pessoas. O RubyMine oferece Git > Pull ou Update Project.

ETAPA 7

Criar uma issue e uma branch de trabalho

No GitHub, crie uma issue para representar uma pequena entrega.

Issue sugerida

```
Título: Documentar como executar o RottenPotatoes

Critérios de aceitação:
- informar como instalar dependências
- informar como preparar o banco
- informar como executar testes
- informar como iniciar o servidor
```

Crie uma branch a partir de main

No RubyMine / GitHub	Comando ou resultado
Widget da branch > New Branch	Nome: docs/instrucoes-execucao.
Checkout branch	Confirme que a nova branch está ativa.
Edite README.md	Atenda aos critérios da issue.

Comandos equivalentes

```
git switch main
git pull
git switch -c docs/instrucoes-execucao
```

A branch isola o trabalho em andamento. main permanece estável até que o pull request seja revisado e integrado.

ETAPA 8

Revisar, commitar e enviar a branch

No README, escreva instruções reais para o projeto. Antes de registrar a mudança, abra o diff no RubyMine.

O aluno deve conseguir responder

[ ] Quais linhas foram adicionadas ou removidas?

[ ] A alteração atende aos critérios da issue?

[ ] Há arquivos não relacionados selecionados?

[ ] A mensagem do commit explica a intenção da mudança?

Mensagem sugerida

```
Documenta execução local do projeto
```

Depois do commit, use Git > Push. No primeiro push da branch, confirme a criação do vínculo com origin/docs/instrucoes-execucao.

Comandos equivalentes

```
git add README.md
git commit -m "Documenta execução local do projeto"
git push -u origin docs/instrucoes-execucao
```

Um bom commit é pequeno, coerente e verificável. Evite mensagens vagas como atualização, mudanças ou trabalho final.

ETAPA 9

Abrir e revisar um pull request

Após o push, abra o pull request pelo aviso do RubyMine, por Git > GitHub > Create Pull Request ou pelo botão Compare & pull request no GitHub.

No RubyMine / GitHub	Comando ou resultado
Base	main: branch que receberá a mudança.
Compare / head	docs/instrucoes-execucao: branch com a proposta.
Title	Documenta execução local do RottenPotatoes.
Description	Resuma a mudança, como testá-la e relacione a issue.

Descrição sugerida

```
## O que mudou
Documenta instalação, banco, testes e servidor.

## Como verificar
Seguir os comandos do README em um ambiente local.

Closes #1
```

Revisão em dupla

[ ] ler a descrição

[ ] abrir Files changed

[ ] conferir clareza e exatidão

[ ] deixar um comentário útil

[ ] aprovar ou solicitar alteração

Pull request não é apenas um botão de merge: ele registra proposta, discussão, revisão e decisão.

ETAPA 10

Responder ao feedback, fazer merge e sincronizar

Se houver solicitação de mudança, não abra outro pull request. Edite a mesma branch, faça novo commit e execute push; o pull request será atualizado automaticamente.

```
git add README.md
git commit -m "Esclarece preparação do banco"
git push
```

Depois da aprovação

No RubyMine / GitHub	Comando ou resultado
GitHub ou janela Pull Requests	Execute Merge ou Squash and merge, conforme orientação do professor.
Delete branch	Remova a branch remota após a integração, se permitido.
RubyMine: checkout main	Volte à branch principal local.
Git > Pull	Traga o merge realizado no GitHub para a main local.

Comandos equivalentes após o merge

```
git switch main
git pull origin main
git branch -d docs/instrucoes-execucao
```

Não continue trabalhando na branch já mergeada. Para uma nova tarefa, sincronize main e crie uma nova branch.

Verificação

[ ] A issue foi encerrada pelo pull request.

[ ] A main local contém a documentação.

[ ] O histórico mostra a integração.

ETAPA 11

Atividade guiada de conflito

Esta atividade deve ser conduzida pelo professor. Um conflito ocorre quando Git não consegue combinar automaticamente alterações concorrentes.

Preparação

- Aluno A e aluno B partem da mesma versão de uma linha do README.
- Cada um altera essa mesma linha de forma diferente em uma branch.
- A primeira mudança é integrada em main.
- A segunda branch tenta incorporar a nova main e encontra conflito.

No RubyMine

Use a ferramenta de merge de três painéis: versão local, resultado e versão recebida. Leia as duas intenções, escreva uma solução coerente no resultado e marque o conflito como resolvido.

```
git status
# resolver o arquivo no editor de merge
git add README.md
git commit
```

Não escolha automaticamente Accept Yours ou Accept Theirs sem entender o conteúdo. Resolver conflito é uma decisão sobre o texto final, não uma disputa entre pessoas.

Após resolver

[ ] O arquivo não contém marcadores <<<<<<<, ======= ou >>>>>>>.

[ ] O conteúdo final preserva as informações necessárias.

[ ] Os testes continuam passando.

ETAPA 12

Histórico, recuperação e entrega

Abra Git > Show History ou a aba Log. Escolha um commit e examine autor, data, mensagem e diff.

No RubyMine / GitHub	Comando ou resultado
Show History	Mostra quando e por que um arquivo mudou.
Annotate / Blame	Relaciona linhas ao commit que as alterou.
Revert Commit	Cria um novo commit que desfaz uma mudança já publicada.
Undo Commit	Útil apenas para o último commit local ainda não enviado.

Evite comandos destrutivos para apagar histórico durante a atividade. Em trabalho compartilhado, prefira revert para desfazer uma mudança publicada.

Checklist de entrega

[ ] Repositório GitHub acessível conforme a regra da turma.
[ ] Nenhum segredo ou banco local publicado.
[ ] Branch main atualizada e testes passando.
[ ] Issue com critérios de aceitação.
[ ] Branch com nome relacionado à atividade.
[ ] Commit com mensagem descritiva.
[ ] Pull request com descrição, revisão e merge.

Evidências para avaliação

URL do repositório, URL do pull request, captura ou demonstração do histórico e explicação oral do fluxo editar -> commit -> push -> pull request -> merge -> pull.

ETAPA 13

Cronograma autoguiado e referências

Plano para a atividade sem acompanhamento presencial

Siga os blocos na ordem indicada e use as etapas anteriores como guia. Ao terminar cada bloco, confira a evidência antes de avançar. O tempo total previsto é de 2 h 45 min.

Tempo	Atividade	Evidência de conclusão
0:00-0:20	Revisar os conceitos, conferir o projeto local e identificar arquivos que não devem ser versionados.	Projeto aberto e lista de exclusões conferida.
0:20-0:45	Inicializar o Git, configurar a identidade e revisar o arquivo .gitignore.	Repositório inicializado e nenhum segredo ou banco local selecionado.
0:45-1:05	Revisar o diff, executar os testes e criar o primeiro commit.	Testes passando, commit criado e área de trabalho limpa.
1:05-1:25	Criar o repositório no GitHub, associar origin e publicar a branch main.	URL do repositório acessível no GitHub.
1:25-1:50	Criar uma issue, abrir uma branch relacionada e realizar a alteração proposta.	Issue criada, branch correta e alteração revisada.
1:50-2:10	Criar um commit descritivo, fazer push da branch e abrir o pull request.	Pull request aberto e relacionado à issue.
2:10-2:30	Revisar o pull request em dupla. Se não houver colega disponível, realizar a autoavaliação com o checklist da etapa 12.	Comentário ou registro da revisão e verificações concluídas.
2:30-2:40	Fazer o merge, atualizar a branch main local e conferir o histórico.	Main sincronizada, histórico correto e testes passando.
2:40-2:45	Responder às perguntas de fechamento abaixo.	Respostas registradas para entrega.

Se surgir um bloqueio

Reserve até 10 minutos para reler a etapa correspondente e conferir a mensagem completa do erro. Se o problema continuar, registre a etapa, o comando ou ação executada, a mensagem de erro e uma captura de tela; publique esse registro no fórum da disciplina no Aprender, para que o monitor possa responder.

Perguntas de fechamento

- Qual é a diferença entre commit e push?
- O que origin representa?
- Por que revisar o diff antes do commit?
- Por que desenvolver em uma branch em vez de diretamente em main?
- O que acontece com um pull request quando um novo commit é enviado à mesma branch?
- Por que executar pull após um merge feito no GitHub?

Referências oficiais

RubyMine - Git tutorial: https://www.jetbrains.com/help/ruby/working-with-git-tutorial.html

RubyMine - GitHub projects: https://www.jetbrains.com/help/ruby/manage-projects-hosted-on-github.html

RubyMine - Pull requests: https://www.jetbrains.com/help/ruby/create-and-merge-github-pull-requests.html

RubyMine - Sync remote: https://www.jetbrains.com/help/ruby/sync-with-a-remote-repository.html

GitHub Docs - Pull requests: https://docs.github.com/en/pull-requests

Git documentation: https://git-scm.com/docs